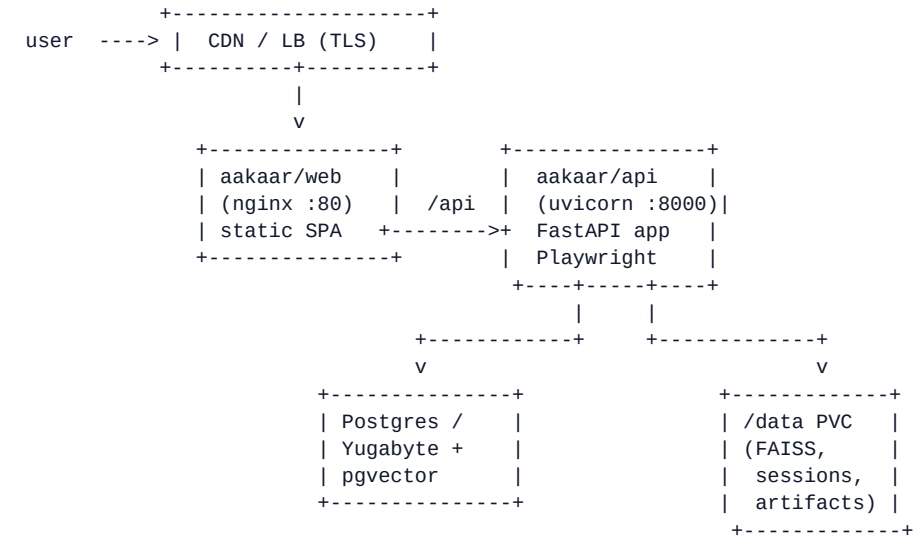


Aakaar — Deployment Architecture & Infra

End-to-end deployment topology for the Aakaar platform: container images, data stores, network boundaries, and a step-by-step rollout for staging and production.

1. System topology

Two container images, one Postgres-compatible store, one persistent volume.



2. Components

Component	Notes
aakaar/web	nginx serving Vite bundle. Stateless. Horizontally scalable.
aakaar/api	FastAPI + Playwright. Stateful in-process (FAISS, browser pool). Run 1 replica per tenant cluster; scale by tenant sharding, not by replica.
Postgres / Yugabyte	System-of-record + pgvector for capability embeddings.
/data PVC	Run artifacts, browser session storage, FAISS index in dev.
LB / CDN	Terminates TLS, fronts the web image. Pass through /api unchanged.
Object store	Not used in v1 — artifacts live on the /data PVC.

3. Local stack via docker compose

docker-compose.yml at the repo root brings up Postgres, the API, and the web image with sensible defaults. Use it for integration testing of the production containers, not for day-to-day dev (the local **start.sh** stays faster).

```
# Local production-like stack. Bring up with:
#   AAKAR_JWT_SECRET=$(openssl rand -hex 32) OPENAI_API_KEY=sk-... docker compose up --build
#
# Brings up:
#   - postgres (Yugabyte/Postgres-compatible store; swap image for production)
#   - aakar-api (FastAPI + Playwright on :8000)
#   - aakar-web (nginx serving the Vite bundle on :5173, /api proxied to aakar-api)
services:
  postgres:
    image: postgres:16-alpine
    environment:
```

```

    POSTGRES_USER: aakar
    POSTGRES_PASSWORD: aakar
    POSTGRES_DB: aakar
volumes:
  - aakar-pg:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U aakar -d aakar"]
  interval: 5s
  timeout: 3s
  retries: 20
ports:
  - "5432:5432"

aakar-api:
  build:
    context: ./aakar
    dockerfile: Dockerfile
  depends_on:
    postgres:
      condition: service_healthy
  environment:
    AAKAR_DB_URL: postgresql+psycopg://aakar:aakar@postgres:5432/aakar
    AAKAR_JWT_SECRET: ${AAKAR_JWT_SECRET:?set AAKAR_JWT_SECRET in env}
    AAKAR_DATA_DIR: /data
    AAKAR_CORS_ORIGINS: http://localhost:5173,http://127.0.0.1:5173
    AAKAR_BROWSER_HEADLESS: "true"
    OPENAI_API_KEY: ${OPENAI_API_KEY:-}
    AAKAR_LLM_MODEL: ${AAKAR_LLM_MODEL:-gpt-4.1-mini}
  volumes:
    - aakar-data:/data
  ports:
    - "8000:8000"
  restart: unless-stopped

aakar-web:
  build:
    context: ./aakar-web
    dockerfile: Dockerfile
  args:
    VITE_API_BASE: /api
  depends_on:
    - aakar-api
  environment:
    AAKAR_BACKEND_URL: http://aakar-api:8000
  ports:
    - "5173:80"
  restart: unless-stopped

volumes:
  aakar-pg:
  aakar-data:

AAKAR_JWT_SECRET=$(openssl rand -hex 32) \
OPENAI_API_KEY=sk-... \
docker compose up --build

```

4. Production rollout

4.1 One-time bootstrap

- Provision a Postgres-compatible cluster (Yugabyte recommended). Enable the **vector** extension on the Aakaar database.
- Provision a persistent volume class for /data. Recommend ReadWriteOnce, 20–50 GiB to start. Snapshot policy: daily, 14-day retention.
- Create a secret store entry for: AAKAR_JWT_SECRET, AAKAR_DB_URL, OPENAI_API_KEY, AAKAR_SUPERUSER_EMAIL, AAKAR_SUPERUSER_PASSWORD.
- Mint TLS certs for app.aakaar.example (web) and optionally api.aakaar.example (only if not co-fronting via /api).
- Push images to a private registry: **aakaar/api:<sha>**, **aakaar/web:<sha>**.

4.2 Per-deploy steps

```
# 1. Build, tag with git sha
GIT_SHA=$(git rev-parse --short HEAD)
docker build -t registry.internal/aakaar/api:$GIT_SHA -f aakar/Dockerfile aakar
docker build -t registry.internal/aakaar/web:$GIT_SHA -f aakar-web/Dockerfile aakar-web
docker push registry.internal/aakaar/api:$GIT_SHA
docker push registry.internal/aakaar/web:$GIT_SHA

# 2. Run migrations as a Job (out-of-band, before rollout)
kubectl run aakar-migrate-$GIT_SHA --rm -i --restart=Never \
  --image=registry.internal/aakaar/api:$GIT_SHA \
  --env=AAKAR_DB_URL=$AAKAR_DB_URL \
  --command -- alembic upgrade head

# 3. Roll the API deployment
kubectl set image deploy/aakar-api api=registry.internal/aakaar/api:$GIT_SHA
kubectl rollout status deploy/aakar-api --timeout=5m

# 4. Roll the web deployment (no migrations, fast)
kubectl set image deploy/aakar-web web=registry.internal/aakaar/web:$GIT_SHA
kubectl rollout status deploy/aakar-web --timeout=2m
```

4.3 Rollback

- Re-run **kubectl rollout undo** on either deployment; both images are immutable by sha.
- Migrations are forward-only by convention. If a migration must be reverted, write an explicit down-migration and run it as a Job before rolling images back.

5. Network & egress

- Inbound: only the LB reaches **aakaar/web**:80. The web image reaches **aakaar/api**:8000 via the cluster network.
- Egress (api): OpenAI API, plus any tenant URLs the planner is allowed to log into. Lock down everything else — Playwright runs untrusted page JS.
- Egress (web): none required at runtime. The CDN fronts assets.
- **AAKAR_CORS_ORIGINS** must list only the production web origin(s). The default config rejects unknown origins.

6. Backups & DR

Asset	Strategy
Postgres / Yugabyte	Continuous WAL + nightly base backup. RPO 5 min, RTO 30 min.
/data PVC	Volume snapshots daily, 14-day retention. Restore by mounting onto a fresh API pod.
Capability registry	Source-controlled in repo (aakar/aakar/shared/registry). Rebuilt on first request after deploy.
Secrets	Managed by the platform secret store; no backup outside that store.

7. CI/CD checklist

- Run **pytest + ruff + mypy** for the backend and **tsc -b** for the frontend on every PR.
- Build both images on merge to **main**, tag with the merge commit sha, push to registry, run a smoke deploy to staging.
- Promote to production by tagging the same image sha into the prod registry path; never rebuild between staging and prod.
- Surface migration plan in the PR description for any change that touches **aakar/db/migrations/** — reviewers must see what will run.

8. Open items deferred to v2

- Object storage for run artifacts — v1 keeps them on the /data PVC. Move to S3-compatible storage when the volume crosses ~100 GiB.
- TemporalExecutor — LocalExecutor ships in v1; Temporal lands later behind the same **Executor** protocol with no API or container changes.
- Per-tenant pod isolation — v1 co-locates tenants in one API replica. Add a tenant-sharding controller before onboarding cohorts that need network-level isolation.